M2：会话、压缩与中断恢复

日志格式、分支投影和提交边界

M2：会话、压缩与中断恢复
- 背景、目标与范围
- 行为规格
  - M2-S01：创建、打开与文件定位
  - M2-S02：单写者锁
  - M2-S03：文件编码与追加
  - M2-S04：读取与损坏
  - M2-S05：分支与 checkout
  - M2-S06：上下文配置和编辑
  - M2-S07：恢复悬空工具调用
  - M2-S08：上下文投影
  - M2-S09：token 预算和分组
  - M2-S10：准备与提交压缩
  - M2-S11：abort 和关闭
- 持久化数据结构
  - Session 公共 API
  - 日志示例
- 详细设计与模式
  - 日志仓库与索引
  - 写入屏障与工作单元
  - 恢复状态机
  - 可测试的端口
- 验收示例与测试矩阵
- 具体投影和压缩示例
  - 分支隔离
  - 保留预算按完整组选择
  - 第二次压缩
  - 编辑与压缩的关系
  - 中断恢复故障表
- 取舍、风险与实施顺序

M2：会话、压缩与中断恢复

属性	内容
状态	设计提案 1.0
前置	M1关键订阅和消息配对
交付	`@local/session`、`--resume`
验收	M2-A01 至 M2-A12

背景、目标与范围

内存消息会随进程退出消失。工具还可能已经修改文件，但其返回值没来得及记下。本模块把完整消息持久化为追加日志，并从日志恢复模型可接受的上下文。

目标包括：线性会话与分支、显式上下文编辑、长会话压缩、损坏检测和未完成调用修补。非目标是恢复流式半条消息、自动重试外部副作用、会话格式迁移和多写者合并。原始历史作为审计材料保留，压缩只改变投影。

行为规格

M2-S01：创建、打开与文件定位

`Session.create({workspace,persist:true})` 生成 UUID sessionId，在总览规定的 workspaceHash 目录准备会话。第一次持久化配置或用户消息前不创建 jsonl；创建后第一行必须是 header。`persist:false` 使用相同语义的内存存储，不创建目录或锁。

`Session.open({workspace,sessionId})` 只允许当前工作区 hash 下的 UUID 文件名；路径穿越或非 UUID 拒绝 INVALID_INPUT。header 的 workspace 必须等于当前 realpath workspace，否则 INVALID_CONFIG。打开不自动启动 run。

所有公共异步变更操作经同一操作互斥器。运行中 prompt、checkout、compact、editContext、reloadContext 拒绝 BUSY；abort/getState/clearQueues 可用。打开后先验证，再修补悬空调用，再 `Agent.replaceContext`；只有这三步成功才对外暴露可用 Session。

M2-S02：单写者锁

在写文件前以 `open(lockPath,'wx',0600)` 创建 `<sessionId>.lock`，内容为 `{pid,hostname,nonce,createdAt}`。占用时拒绝 BUSY，不自动删锁。正常 close 只删除 nonce 与自身匹配的锁。崩溃遗留锁由用户确认原进程已退出后显式移除；程序给出路径，不以 PID 可复用的探测结果自动抢占。

持久化 Session 在 close 前持有锁。只读查看可以不取锁，但不得恢复运行、修补或追加。此设计不支持 NFS 上的可靠跨主机锁，不把它作为验收环境。

M2-S03：文件编码与追加

UTF-8，每个 JSON 对象一行，以 LF 结尾。header 格式版本为 1，后续是 entry 或 checkout 控制记录。每条 entry 有唯一 UUID、parentId 和 timestamp。parentId 是追加时活动叶节点，首条为 null；配置和压缩也是 entry，会推进叶节点。

写入先构造并验证完整字符串，再通过单一写入队列追加。只有完整写成功且必要同步完成后才更新内存 Map 和 leaf。写入失败将 Session 标记 faulted，停止新运行，以 PERSISTENCE 拒绝；不能继续使用内存中未落盘的“成功”状态。

message_update 不写盘；message_end 写一条 message entry。带工具调用的 assistant 以及每个 tool result 在关键监听器中 `datasync` 后返回；普通消息和配置至少在该 run 的 agent_end 或显式操作返回前 datasync。close 等待队列、同步并关闭 fd。不可依赖操作系统一次 write 必然写完，需处理短写。

M2-S04：读取与损坏

先按字节找到最后一个 LF。LF 后的非空尾部视为未提交数据，返回 warning，持锁以原偏移 truncate 后才可追加。即使它恰好能 JSON.parse，也不是 committed。末尾完整 LF 行若 JSON 无法解析则视为损坏，不能直接跳过。

从 header 起逐行解析和 schema 校验：未知版本、重复 ID、未知记录类型、缺父节点、父节点在当前记录之后、无效 checkout target、跨分支引用、非法消息结构均 CORRUPT_SESSION，打开失败。错误带行号，不自动删中间坏行。空行不是合法记录。若仅有合法 header，允许恢复为空会话。

加载时逐条更新活动 leaf：entry 总是接到当前 leaf；checkout 只改 leaf。文件是追加顺序，父只能指向已出现 entry，所以无需一般图的拓扑排序。还应检测每个分支路径上的消息配对；唯一允许缺失的是活动或可切换分支尾部的未完成工具批，按 S07 修补。

M2-S05：分支与 checkout

`checkout(entryId|null)` 仅 idle，target 必须存在；null 表示 header 后的空分支。验证目标路径的投影合法，追加一个 checkout 控制记录并同步后更新 leaf。下一 entry 的 parentId 指向 target，从而产生新分支。旧分支条目不删除。

checkout 记录不是 entry，没有 parentId，不参与祖先链。重新打开按文件序处理 checkout，恢复的是最后选中的分支。只在内存修改 leaf 会丢失选择，因此禁止。

切到工具批不完整的位置时先执行 S07 修补，再注入 Core。`getBranch(leafId=currentLeaf)` 返回从根到目标的 entry 快照；`listBranches()` 返回所有没有子 entry 的叶节点与当前 leaf（若非叶也加入），附最后一条用户文本的最多80字符预览。两者只读，不写 checkout，未知 ID 为 NOT_FOUND。列表按最后 entry timestamp、再 ID 排序；当前分支清楚标记。空会话返回一个 leafId=null 的分支。

切换不改变磁盘工作区，不运行 Git checkout，不撤销文件修改。CLI 应明确显示“切换会话分支”，不暗示文件已经恢复。

M2-S06：上下文配置和编辑

`setContext({systemPrompt,toolsFingerprint,resourceFingerprint,model})` 仅 idle。配置记录持有完整快照，不能只保存“增加一个技能”这种依赖当前文件的操作。投影使用活动分支上最后一条 context entry，若无则用创建时默认模型和空 prompt；实际工具实例由 CLI 根据配置重建，不能反序列化函数。

`editContext(targetMessageEntryId,replacementText|null)` 仅 idle，只允许修改当前分支、当前压缩保留范围内的普通 user 或无调用 assistant 文本消息。不能编辑工具调用、工具结果、配置、摘要或其他 edit。replacement 非空字符串表示替换文本，null 表示从模型投影移除；空白字符串拒绝。追加 edit entry，不更改目标原行。同目标最后一条编辑生效。目标在压缩已吸收的前缀中则 CONFLICT，应切换至压缩前分支再编辑，防止旧摘要偷偷保留被删除内容。

M2-S07：恢复悬空工具调用

按活动分支原始消息顺序扫描。每条含调用的 assistant 建立 callId 到工具名映射；之后 tool result 必须匹配且仅出现一次。未知或重复结果是 CORRUPT_SESSION。缺结果仅可出现在分支消息尾部：例如 assistant 有 A/B，后面有 A 结果但没有 B，且尚无下条 user/assistant。若历史中间缺结果却已有后续模型消息，则拒绝，不猜测。

为每个缺失调用按原调用顺序追加 toolResult：`isError:true`，code=`ABORTED`，details=`{recovered:true,outcomeUnknown:true}`，content 为 `Execution outcome unknown after interruption; not retried.`。结果说明的是“未知”，即使找到目标文件也不能认定调用成功。恢复不执行工具、不询问模型。

修补和 datasync 成功后再注入 Core。重开时发现已有对应结果，不再次追加。进程若在多条修补之间再次退出，下一次只补剩余项。只读查看可以展示缺失诊断，但不修改文件。

M2-S08：上下文投影

`project(entries,leaf): SessionProjection` 是纯函数。步骤顺序固定：

1. 从 leaf 沿 parentId 回溯至 null，翻转为时间顺序，验证路径。
2. 从该路径提取最后的 context 快照。
3. 找最后一条 compaction；没有则取全部 message。
4. 有 compaction 时，生成单条摘要 user 消息，然后取该记录指向的 firstKeptEntryId 起的 message，包括压缩记录之后的新消息。
5. 应用可见 edit，删除 replacement=null 的消息，保持其他消息的 ID 和顺序。
6. 确认工具调用与结果配对完整，返回 messages、systemPrompt、model、leaf 和版本指纹。

摘要消息使用确定性 ID `summary:<compactionId>`，timestamp 为压缩记录时间，text 为 `[Conversation summary]\n` 加摘要；它只在投影中存在，不再作为 message entry 追加。给摘要包前缀有助于区分用户新请求，但不是安全权限边界。

M2-S09：token 预算和分组

定义教学用估计函数：每条消息 `4 + ceil(UTF8Bytes(canonicalPayload)/4)`，system prompt、工具声明分别同法估计；canonicalPayload 使用固定字段序和无空格 JSON 序列化。数值不宣称是供应商真实 tokenizer。

`shouldCompact = estimatedTokens > contextWindow - reserveTokens`，等号不触发。自动压缩只在 idle、接受新输入之前进行；预算计算应包含将要提交的用户文本和当前工具声明。收到工具结果后若下一请求仍超预算，本次 run 以 INVALID_INPUT 结束，用户可在 idle 压缩后继续；不在执行中的 run 内偷偷修改历史。

保留单位为“对话组”：一条 user 起，到下一条 user 前的全部 assistant/toolResult。开头若有无 user 的 assistant（如 continue），它们合成一个前置组；摘要作为特殊前缀。不能把 assistant call 和其 result 拆开。context/edit/compaction 不是消息组的成员。

M2-S10：准备与提交压缩

`compact({instructions?,signal?})` 仅 idle，通过 Session 的操作锁排除 prompt 和 checkout。取 leaf 和投影快照，至少要有两个非摘要对话组。自末尾向前保留完整组，直到累计消息估计达到 keepRecentTokens；至少保留最新组。剩下的前缀才可摘要；若没有剩余组，返回 `{status:'no_change'}`，不调用摘要函数、不追加记录。

`prepare` 产出 prefix、keptIds、firstKeptEntryId、baseLeaf 和 tokensBefore。prefix 包括先前摘要，以便多次压缩延续旧信息。firstKeptEntryId 必须是当前分支祖先中的 message entry，不能指向工具结果或虚拟 summary。

摘要器通过独立 `SummarizeFn` 注入，不调用 Agent.prompt，不执行工具，不污染主会话。传入 prefix 和 instructions，返回非空 string。建议摘要内容涵盖任务、已做修改、关键路径、约束和待办；不要求测试匹配生成文案。

返回后检查取消、leaf 未变、schema 和新预算。新投影仍超过 contextWindow-reserveTokens 则 INVALID_INPUT，不追加；不能无限重试。合法时追加一个 compaction entry 并同步，再替换 Core 上下文。失败、空摘要、取消或 baseLeaf 变化均无 compaction 记录，旧投影保持不变。

M2-S11：abort 和关闭

`session.abort()` 调用 Core.abort 并等待 idle 和写入队列；队列消息不清。`clearQueues()` 返回未消费项，不写日志。正常 close 在 abort/idle 后同步并释放锁。close 后任何变更 API 拒绝 INVALID_INPUT；多次 close 幂等。

磁盘工具操作和日志结果不是一个事务。若工具已写文件而记录结果失败，Session faulted，下一次打开执行 S07，不自动重复写文件。这是恢复语义的必要限制。

持久化数据结构

```
type Header = {
  type: "session";
  version: 1;
  sessionId: string;
  workspace: string;
  createdAt: number;
  defaultModel: Model;
};
type EntryBase = {
  id: string;
  parentId: string | null;
  timestamp: number;
};
type MessageEntry = EntryBase & {
  type: "message"; message: AgentMessage;
};
type ContextEntry = EntryBase & {
  type: "context";
  systemPrompt: string;
  toolsFingerprint: string;
  resourceFingerprint: string;
  model: Model;
};
type EditEntry = EntryBase & {
  type: "context_edit";
  targetId: string;
  replacement: string | null;
};
type CompactionEntry = EntryBase & {
  type: "compaction";
  summary: string;
  firstKeptEntryId: string;
  tokensBefore: number;
  tokensAfter: number;
};
type SessionEntry = MessageEntry | ContextEntry | EditEntry | CompactionEntry;
type CheckoutRecord = {
  type: "checkout";
  targetId: string | null;
  timestamp: number;
};
type FileRecord = Header | SessionEntry | CheckoutRecord;
```

entry.id 与 message.id 属于不同命名空间，不要求相同。callId 在同一会话唯一，生成结果消息引用 callId，不引用 entry.id。edit 和 firstKeptEntryId 引用 entry.id，因此即便投影改文本仍能定位原始行。

header 仅描述创建时默认值，运行时模型变化由 context entry 表示。toolsFingerprint 是工具声明规范 JSON 的 SHA-256，不用于还原工具函数。恢复时若工具集与快照不同，CLI 必须在新输入前追加新的 context，旧快照留在历史。

Session 公共 API

```
type SessionOptions = {
  workspace: string;
  agent: AgentPort;
  defaultModel: Model;
  persist?: boolean;
```

```
  summarize: SummarizeFn;
  settings: { reserveTokens: number; keepRecentTokens: number };
};
type CompactionPreparation = {
  baseLeaf: string;
  prefix: readonly AgentMessage[];
  keptEntryIds: readonly string[];
  firstKeptEntryId: string;
  tokensBefore: number;
};
type SummarizeFn = (messages: readonly AgentMessage[], options: {
  instructions?: string;
  signal: AbortSignal;
}) => Promise<string>;
type SessionProjection = {
  leafId: string | null;
  messages: AgentMessage[];
  systemPrompt: string;
  model: Model;
  resourceFingerprint: string;
  toolsFingerprint: string;
};
interface SessionPort {
  prompt(text: string, options?: { runId?: string }): Promise<RunResult>;
  continue(options?: { runId?: string }): Promise<RunResult>;
  abort(): Promise<void>;
  getBranch(leafId?: string | null): readonly SessionEntry[];
  listBranches(): readonly { leafId: string | null; preview: string; current: boolean }[];
  checkout(entryId: string | null): Promise<void>;
  editContext(entryId: string, replacement: string | null): Promise<void>;
  setContext(context: Omit<ContextEntry, keyof EntryBase | "type">): Promise<void>;
  compact(options?: { instructions?: string; signal?: AbortSignal }):
    Promise<{ status: "compacted"; entryId: string } | { status: "no_change" }>;
  clearQueues(): { steering: QueueItem[]; followUp: QueueItem[] };
  close(): Promise<void>;
}
```

Session.create/open 为异步工厂，完整校验锁和日志后返回实现 SessionPort 的对象。`getState()` 返回 Gateway 页中所需的只读状态，`subscribe()` 转发 Core 事件和 session operation 事件；不能以此 API 从 Gateway 直接 append 任意 entry。`steer/followUp` 只转发 Core 队列接口，不持有第二份队列。

ContextEntry 中 model/systemPrompt 是序列化值；toolsFingerprint 不能还原可执行工具。打开会话时应先由 CLI 提供实际 tools，再投影消息；MCP 缺失也可恢复历史，只是新调用对应工具会失败。恢复旧消息不能重新执行历史工具来“找回工具实例”。

日志示例

下例为阅读方便以短 ID 表示 UUID；真实文件必须使用 UUID。

```
{"type":"session","version":1,"sessionId":"s","workspace":"/work","createdAt":1,"defaultModel":{"id":"demo","prov
ider":"demo","contextWindow":8192}}
{"type":"message","id":"e1","parentId":null,"timestamp":2,"message":{"id":"m1","timestamp":
2,"role":"user","text":"读文件"}}
{"type":"message","id":"e2","parentId":"e1","timestamp":3,"message":{"id":"m2","timestamp":3,"role":"assistant","
text":"","calls":[{"callId":"c1","name":"read","arguments":{"path":"a.txt"}}],"stopReason":"toolUse"}}
{"type":"message","id":"e3","parentId":"e2","timestamp":4,"message":{"id":"m3","timestamp":
4,"role":"toolResult","callId":"c1","toolName":"read","content":"hello","isError":false}}
{"type":"checkout","targetId":"e1","timestamp":5}
```

{"type":"message","id":"e4","parentId":"e1","timestamp":6,"message":{"id":"m4","timestamp":6,"role":"user","text":"换个问题"}}

重新加载 leaf=e4，当前消息只有 m1、m4。e2/e3 保留在另一分支。

详细设计与模式

日志仓库与索引

`SessionStore` 拥有 fd、lock、append queue、faulted 状态和字节偏移。`SessionManager` 拥有 `Map<entryId,SessionEntry>`、`children:Map<parentId,string[]>`、leaf 和 header；不执行模型。Session 作为 Facade 协调 Core、Store 和压缩器。

载入 n 条记录 O(n)，空间 O(n)；append 平均 O(1) 加内容字节数；回溯深度 h 的分支 O(h)；用 Map 预收集最后一次 edit，使投影 O(h + 消息字节数)，不能为每个目标重新扫描整个日志形成 $O(h^2)$。children 用于列分支，可推迟到需要时生成。

追加事件日志和投影类似 CQRS 的读写分离：写侧只追加事实，读侧构建模型上下文。此项目不要求独立数据库、消息总线或完整 CQRS 框架。磁盘文件是权威，Core transcript 是可丢弃缓存。

写入屏障与工作单元

Session 在创建 Core 其他订阅前注册关键监听器。每个 message_end 形成一个工作单元：验证 → 序列化 → 完整 append → 需要时 datasync → 更新内存索引 → 返回。工具执行在此之后开始，体现写前记录思想。

压缩的工作单元是一个 compaction entry，而不是把所有旧消息删掉。摘要计算在提交前，可取消；提交后旧 leaf 移到新记录，投影切换是一致的。锁内 await 摘要期间其他变更返回 BUSY，不让多个摘要竞争提交。

恢复状态机

状态	允许操作	转移
opening	验证、修补	成功→idle；错误→closed
idle	prompt、checkout、compact、editContext、close	对应工作态
running	abort、clearQueues、getState	Core 结束且 flush→idle
compacting	abort、getState	提交/取消→idle
faulted	getState、close	禁止再次 prompt
closed	重复 close	其余拒绝

abort 在 compacting 时取消摘要 signal；不调用当前不存在的 Core run。Session 不能只看 Core.isBusy 判断自身是否可变更，因为压缩和日志提交也占用操作锁。

可测试的端口

`StorageIO` 封装 append/sync/truncate，允许测试在第几次 write 注入错误或短写。`SummarizeFn(prefix,{instructions,signal})` 是策略端口。Clock 和 ID 工厂让日志夹具可重现。注入只到边界，不给每个私有函数加 mock 开关。

验收示例与测试矩阵

编号	设置与操作	预期
M2-A01	create 后不写；再 prompt	首次之前无 jsonl；header 第一行；权限正确
M2-A02	三条消息追加再 open	parent 链、ID、模型上下文与原会话一致
M2-A03	文件尾加半个 JSON/合法 JSON 但无 LF	仅尾部丢弃并 truncate；可继续追加；旧行不丢
M2-A04	中间坏 JSON、重复 ID、缺父、未知 version	CORRUPT_SESSION；不改文件
M2-A05	checkout 历史节点后追加；关闭再打开	原分支保留；leaf 是新分支；checkout 持久化
M2-A06	assistant A/B，已有 A result	只补 B，details.outcomeUnknown=true；重开不重复
M2-A07	工具 assistant 落盘前注入 append/sync 错误	PERSISTENCE；execute=0；Session faulted
M2-A08	工具写文件后、result 落盘前失败	文件保留；恢复不再调用工具；补未知结果
M2-A09	edit user 两次并删除	最后 edit 生效；原行不变；工具消息 edit 拒绝
M2-A10	contextWindow=1000,reserve=100，估计900/901	等号不触发，901触发；组内调用与结果不分割
M2-A11	压缩成功/摘要失败/取消/无可压缩前缀	仅成功追加一行；旧历史仍在；重开投影一致
M2-A12	两个写者、close、运行中 checkout	第二写者 BUSY；正常释放锁；活动运行不受拒绝操作影响

额外边界：最近组本身超预算、两次压缩、切到另一分支后摘要隔离、对已压缩前缀编辑拒绝、header-only 会话。验证“保留全文”要同时检查 JSONL 原字节和投影，不能只看 UI。

具体投影和压缩示例

分支隔离

设 entry 链为 e1(user)→e2(assistant stop)→e3(user)→e4(assistant stop)。checkout(e2) 后追加 e5(user)。当前消息应来自 e1/e2/e5，不包含 e3/e4。再次 checkout(e4) 会恢复原来的四条消息。checkout 记录只控制活动指针，所以不会出现在 prompt 里。

如果在 e5 分支生成 compaction，它只影响该路径；原 e4 分支没有这条 compaction 祖先，投影不使用它。以“文件中最后一条摘要”代替“当前路径最后一条摘要”会错用其他分支内容，必须有专门验收夹具。

保留预算按完整组选择

假设四个对话组 G1/G2/G3/G4，估计分别为80、120、90、70；keepRecentTokens=150。自末尾取G4=70，再取G3达到160，保留G3+G4，摘要G1+G2。firstKeptEntryId 是G3首个user的 entry ID。若G3有assistant调用和两个tool result，全部保留，不能为贴合150硬拆。

若最新组G4本身为300而目标为150，仍完整保留G4；目标是软预算。压缩前缀可为空则 no_change；若压缩后仍超模型可用窗口，操作失败而不追加摘要。调用方应告诉用户换更大窗口或另开会话，不能死循环调用摘要器。

第二次压缩

第一次投影为summary(C1)+G3+G4。之后追加G5/G6，第二次保留G5/G6，前缀应包含 summary(C1)+G3+G4，生成C2。最新投影为summary(C2)+G5+G6，不能同时塞两份 summary(C1)/summary(C2)。文件仍有所有旧entry和C1，切回旧叶节点仍能得到旧投影。

编辑与压缩的关系

先edit用户文本，再compact时，摘要器必须读到编辑后的版本。压缩后编辑已吸收的目标拒绝 CONFLICT。投影应用edit时只处理当前保留消息的entry ID；指向已被摘要吸收的合法历史edit 不再重复应用到虚拟摘要。不能用字符串替换方式“修改摘要里的旧句子”，这会引入不可验证的语义猜测。

中断恢复故障表

中断时刻	磁盘可能情况	打开后的行为
assistant delta 中	没有完整assistant行	保留已提交user；不恢复半条；不执行工具
assistant行写入但未LF	未提交尾部	截掉尾部；Core此前不应执行工具
assistant调用已同步，execute前	有call无result	补未知结果，不execute
execute完成，result未提交	文件/远端可能已改变	同样补未知，不重试
第一条修补完成、第二条未完成	一部分配对	只补剩余项

中断时刻	磁盘可能情况	打开后的行为
compact摘要计算中	无compaction行	旧投影不变
compaction行已同步	有新摘要record	使用最新摘要和保留范围

流式文字不是恢复材料，UI 曾显示什么不能改变以上规则。正常abort会尽量提交终态，强杀进程则依赖日志前缀和修补。

取舍、风险与实施顺序

简单跳过所有坏行会隐藏断裂父链，因此只丢未提交尾部；中间损坏明确报错。自动重试缺结果调用看似便利，却可能重复扣款或覆盖文件，因此恢复只补未知结果。同步每个关键消息增加 I/O 延迟，换来清晰的执行边界；流式 delta 不同步以控制成本。

先实现内存 entry 树与投影，再加日志编解码和错误注入，随后接关键订阅、悬空调用修补，最后做分组压缩和分支操作。先让重开后的消息完全一致，再加入压缩，否则难以区分恢复和摘要错误。